

PRÁCTICA SLEEP MODE

Modos de ahorro de energía en ESP32-C6 utilizando micropython

Sistemas Embebidos

MicroPython · Sleep Mode

1. Objetivo

En esta práctica se busca entender cómo funciona el ahorro de energía en el ESP32-C6, específicamente usando los modos **Light Sleep y Deep Sleep**.

La idea es ver cómo el microcontrolador puede “descansar” cuando no está trabajando y cómo puede volver a activarse usando diferentes métodos de despertado (wake up), como un temporizador o una señal por GPIO.

Todo esto se implementa usando **MicroPython en Thonny IDE**, para observar de forma práctica cómo cambia el comportamiento del sistema cuando entra en estos modos de bajo consumo.

También se busca comprender cómo estos modos se utilizan en sistemas reales como IoT o dispositivos a batería.

1.1 Objetivos Específicos

- Entender cómo funcionan los modos de bajo consumo del ESP32-C6.
- Diferenciar entre Light Sleep y Deep Sleep de forma práctica.
- Configurar el despertar del sistema por tiempo (timer).
- Usar un botón como método de wake up.
- Observar qué pasa con el sistema cuando entra en reposo.

1.2 Alcance de la Práctica

Esta práctica se centra en implementar los modos de ahorro de energía del ESP32-C6 y analizar cómo se comporta el sistema en cada uno.

Se busca ver claramente la diferencia entre un modo donde el sistema solo pausa la ejecución (Light Sleep) y otro donde prácticamente se reinicia (Deep Sleep).

2. Hardware Utilizado

2.1 Componentes

Componente	Cantidad	Descripción
ESP32-C6	1	Microcontroladores principales con soporte WiFi y ESP-NOW
Cable USB-C	1	Programación y alimentación
PC con Thonny IDE	1	Entorno de desarrollo y consola serial
Cable de protoboard	1	Conexiones eléctricas entre módulos
Proto Board	1	Conexiones
Push Button	1	Nos ayudara a despertar por interrupción externa

3. Marco teórico

3.1 Modos de bajo consumo en ESP32-C6

El ESP32-C6 tiene diferentes estados de energía que le permiten ahorrar batería cuando no está haciendo tareas importantes. Existen dos tipos de modos de ahorro de energía principales modo Light Sleep, Deep Sleep, dependiendo del tipo de aplicación se puede configurar otros tipos de ahorro de energía, donde el consumo de energía se disminuye debido al voltaje, además de utilizar su propio bus de comunicación avanzado para una comunicación más eficiente también llamado Advanced Peripheral Bus (APB). Además de incluir un procesador de bajo consumo LP CPU (Low Power CPU) y de incluir una velocidad de reloj adicional (RTC) de 20MHz.

Algunas consideraciones para conocer son:

- Todas las partes del ESP32-C6 se apagan exceptuando:
 - Controlador RTC
 - Coprocesador ULP
 - Memoria RTC FAST
- Para las formas de despertar se puede utilizar la interfaz de programación de aplicaciones (Application Programming Interface).
- Al despertar nuestro microcontrolador se reinicia.
- Los modos de WiFi y Bluetooth se desactivan al iniciar alguno modo Sleep.

3.1.1 Light Sleep

En este modo el microcontrolador baja su consumo al desactivar algunas partes.

- La CPU se detiene
- La RAM sigue activa
- El sistema puede despertar rápido
- Consume menos energía, pero no es el mínimo posible

3.1.2 Deep Sleep

En este tipo de modo de apagado desactiva la gran mayoría de las funciones esenciales de nuestro microcontrolador, aquí una consideración importante es que mantiene algunos GPIO activados o desactivados, dependiendo de cómo se configuro.

Aquí prácticamente todo se apaga:

- La CPU deja de funcionar
- Solo mantiene la memoria RTC/LP CPU
- El consumo baja muchísimo
- Al despertar, el sistema se reinicia

3.1.3 WiFi

En este modo de ahorro de energía, el consumo se reduce al minimizar el tiempo en que el módulo WiFi permanece activo. El sistema solo activa la radio durante los momentos necesarios para la recepción o transmisión de tramas de datos, evitando así el monitoreo constante del canal.

Es importante recalcar que este modo puede trabajar en conjunto con los estados de **Light Sleep** y **Deep Sleep**, dependiendo de la configuración del sistema y la aplicación.

3.1.4 Bluetooth

En el caso de Bluetooth, el ahorro de energía se logra reduciendo la actividad del radio y controlando los periodos en los que el dispositivo permanece visible o conectado.

El sistema puede permanecer en estados de bajo consumo mientras mantiene conexiones BLE (Bluetooth Low Energy), activándose únicamente cuando es necesario enviar o recibir datos.

De igual manera, este modo puede integrarse con **Light Sleep** y **Deep Sleep**, permitiendo que el microcontrolador reduzca su consumo energético sin perder la capacidad de comunicación inalámbrica cuando se requiera.

Es importante recalcar que nos centraremos en los modos Deep Sleep y Light Sleep, los modos WiFi, y Bluetooth se desactivan cuando se activa alguno de los modos Deep Sleep o Light Sleep, con lo cual los modos WiFi, y Bluetooth **NO SE ENCUENTRAN OPTIMIZADOS EN MICROPYTHON**.

3.2 Formas de despertar (Wake Up)

Dependiendo del tipo de modo de suspensión, se pueden utilizar diferentes formas para despertar el microcontrolador. Entre las más comunes destacan:

- **Timer:** permite despertar el microcontrolador después de que ha transcurrido un tiempo determinado.
- **Cambio lógico externo:** el microcontrolador se activa mediante un flanco de subida, bajada o cualquier cambio en el estado lógico de un pin.

Light Sleep

En este modo de bajo consumo, el sistema aún mantiene más periféricos activos, por lo que existen varias formas de despertar el microcontrolador:

- **Timer:** despierta el sistema después de un tiempo configurado.
- **GPIO:** permite configurar pines individuales para detectar cambios de estado lógico (alto o bajo).
- **UART:** el microcontrolador puede despertarse al recibir datos a través de la comunicación serial.

Deep Sleep

En este modo el consumo es mucho menor, por lo que las opciones de despertar son más limitadas:

- **Timer:** despierta el sistema después de un tiempo definido.
- **GPIO:** permite despertar mediante un cambio de estado en un pin específico.
- **IO / RTC GPIO:** detecta cambios de nivel lógico en pines configurados con resistencia Pull-Up o Pull-Down durante el estado de Deep Sleep.

3.3 Arquitectura de bajo consumo

El ESP32-C6 cuenta con una arquitectura diseñada para reducir al máximo el consumo de energía, especialmente en aplicaciones IoT o sistemas a batería. Dentro de esta arquitectura destaca el **Low-Power CPU (LP CPU)**, que es un procesador de 32 bits basado en RISC-V. Este procesador es de bajo consumo y trabaja con una estructura simple de ejecución, lo que le permite funcionar de manera eficiente cuando el sistema principal no está activo.

4. Descripción Técnica Detallada

4.1 Inicialización del sistema de bajo consumo (Light Mode)

El sistema de bajo consumo en el ESP32-C6 inicia configurando el modo de suspensión que se va a utilizar.

Dependiendo del modo seleccionado, el comportamiento del sistema cambia:

En Light Sleep, la CPU se detiene pero la memoria RAM sigue activa.

En Deep Sleep, la mayoría del sistema se apaga y solo permanece activa la parte RTC/LP encargada del despertar.

4.2 Configuración del sistema de bajo consumo

El ESP32-C6 maneja los modos de ahorro de energía a través del módulo machine en MicroPython.

El microcontrolador puede entrar en estado de suspensión utilizando funciones específicas que controlan el tiempo de apagado del sistema.

El sistema puede despertar mediante:

- Temporizador (RTC timer)
- Eventos de hardware compatibles

4.3 Lógica principal del sistema — Sleep Mode.py

Este es el archivo principal donde se ejecuta el ciclo de funcionamiento del sistema en modos de bajo consumo.

El programa permite observar cómo el ESP32-C6 entra en estado de reposo y posteriormente se reactiva.

```
import machine
import time

print("Sistema activo")

while True:

    print("Ejecutando tareas")

    time.sleep(2) #DEJAMOS UN DELAY PARA MOSTRAR MENSAJE
```

```
print("Entra modo light sleep")

machine.lightsleep(5000)

#EL ESP32 SE REINICIA AL SALIR DE CUALQUIER MODO SLEEP.
#DEBIDO A ESO ESTE MENSAJE NUNCA DEBE DE APARECER
print("Mensaje que no debe de aparecer en consola")
```

4.4 Explicación del flujo del código

4.4.1 Ejecución del sistema

El programa inicia mostrando un mensaje en consola, indicando que el sistema está activo. Posteriormente entra en un ciclo infinito (Light Sleep) o en una ejecución única (Deep Sleep), dependiendo del modo seleccionado.

4.4.2 Light Sleep

Se visualiza a través de la consola que nuestro ESP32 sigue en su modo normal.

```
print("Sistema activo")
```

Entramos a un ciclo que solo se repetirá una vez debido a que el modo light sleep hará que se reinicie al terminar el tiempo.

Además dejamos un instante para poder visualizar los datos en consola de aproximadamente de 2s.

```
while True:

    print("Ejecutando tareas")

    time.sleep(2) #DEJAMOS UN DELAY PARA MOSTRAR MENSAJE
```

Ahora ejecutamos la función que nos permitirá entrar en el modo light sleep durante 5 segundos después de ello es necesario volver a configurar nuestro ESP32, cabe resaltar que el mensaje final nunca aparecerá debido al reinicio de nuestro microcontrolador.

```
print("Entra modo light sleep")

machine.lightsleep(5000)

#EL ESP32 SE REINICIA AL SALIR DE CUALQUIER MODO SLEEP.
#DEBIDO A ESO ESTE MENSAJE NUNCA DEBE DE APARECER
print("Mensaje que no debe de aparecer en consola")
```

```

1 import machine
2 import time
3
4 print("Sistema activo")
5
6 while True:
7
8     print("Ejecutando tareas")
9
10    time.sleep(2) #DEJAMOS UN DELAY PARA MOSTRAR MENSAJE
11
12    print("Entra modo light sleep")
13
14    machine.lightsleep(5000)
15
16    #EL ESP32 SE REINICIA AL SALIR DE CUALQUIER MODO SLEEP.
17    #DEBIDO A ESO ESTE MENSAJE NUNCA DEBE DE APARECER
18    print("Mensaje que no debe de aparecer en consola")
19

```

Consola x

>>> %Run -c \$EDITOR_CONTENT

```

MPY: soft reboot
Sistema activo
Ejecutando tareas
EntraESP-ROM:esp32c6-20220919
Build:Sep 19 2022
rst:0x10 (LP_WDT_SYS),boot:0xc (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:2
load:0x40875730,len:0xdbc
load:0x4086b910,len:0xc34
load:0x4086e610,len:0x2e3c
entry 0x4086b910
MicroPython v1.28.0 on 2026-04-06; ESP32C6 module with ESP32C6
Type "help()" for more information.

```

4.4.2 Deep Sleep

Se visualiza a través de la consola que nuestro ESP32 sigue en su modo normal.

```
print("Sistema activo")
```

Entramos a un ciclo que solo se repetirá una vez debido a que el modo Deep sleep hará que se reinicie al terminar el tiempo.

además, dejamos un instante para poder visualizar los datos en consola de aproximadamente de 2s.

```

while True:

    print("Ejecutando tareas")

    time.sleep(2) #DEJAMOS UN DELAY PARA MOSTRAR MENSAJE

```

Ahora ejecutamos la función que nos permitirá entrar en el modo light sleep durante 5 segundos después de ello es necesario volver a configurar nuestro ESP32, cabe resaltar que el mensaje final nunca aparecerá debido al reinicio de nuestro microcontrolador.

```
print("Entra modo Deep sleep")
```

```
machine.deepsleep(5000)

#EL ESP32 SE REINICIA AL SALIR DE CUALQUIER MODO SLEEP.
#DEBIDO A ESO ESTE MENSAJE NUNCA DEBE DE APARECER
print("Mensaje que no debe de aparecer en consola")
```

```
1 import machine
2 import time
3
4 print("Sistema activo")
5
6 while True:
7
8     print("Ejecutando tareas")
9
10    time.sleep(2) #DEJAMOS UN DELAY PARA MOSTRAR MENSAJE
11
12    print("Entra modo deep sleep")
13
14    machine.deepsleep(5000)
15
16    #EL ESP32 SE REINICIA AL SALIR DE CUALQUIER MODO SLEEP.
17    #DEBIDO A ESO ESTE MENSAJE NUNCA DEBE DE APARECER
18    print("Mensaje que no debe de aparecer en consola")
19
```

Consola x

>>> %Run -c \$EDITOR_CONTENT

```
MPY: soft reboot
Sistema activo
Ejecutando tareas
Entra modo deep sleep
ESP-ROM:esp32c6-20220919
Build:Sep 19 2022
rst:0x5 (SLEEP_WAKEUP),boot:0xc (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:2
load:0x40875730,len:0xdbc
load:0x4086b910,len:0xc34
load:0x4086e610,len:0x2e3c
entry 0x4086b910
MicroPython v1.28.0 on 2026-04-06; ESP32C6 module with ESP32C6
Type "help()" for more information.
```


5. Flujo del sistema.

A continuación, se muestra el flujo de nuestro sistema del modo deep sleep.

```
INICIO → Importar módulos → Inicializar ESP32
↓
Mostrar mensaje "Sistema activo"
↓
Esperar 2-3 segundos para visualizar en consola
↓
Mostrar mensaje "Entrando en Deep Sleep"
↓
Ejecutar función: machine.deepsleep(5000)
↓
EL ESP32 SE APAGA COMPLETAMENTE
↓
CPU detenida / RAM borrada
↓
RTC espera el tiempo configurado (5 segundos)
↓
EL SISTEMA SE REINICIA AUTOMÁTICAMENTE
↓
El código después de deepsleep() nunca se ejecuta
```

A continuación, se muestra el flujo de nuestro sistema del modo light sleep.

```
INICIO → Importar módulos → Inicializar ESP32
↓
Mostrar mensaje "Sistema activo"
↓
BUCLE PRINCIPAL (while True)
↓
Mostrar mensaje "Ejecutando tareas"
↓
Esperar 2 segundos (time.sleep)
↓
Mostrar mensaje "Entrando en Light Sleep"
↓
Ejecutar función: machine.lightsleep(5000)
↓
EL MICROCONTROLADOR ENTRA EN PAUSA TEMPORAL
↓
Esperar 5 segundos en bajo consumo
↓
EL SISTEMA SE REINICIA AUTOMÁTICAMENTE
↓
El código después de lightsleep() nunca se ejecuta
```

6. LOW POWER — CONCEPTOS CLAVE

La tecnología Low Power en el ESP32-C6 es un conjunto de mecanismos que permiten reducir el consumo energético del microcontrolador mediante la desactivación parcial o total de sus componentes internos. En esta práctica se utilizan principalmente los modos Light Sleep y Deep Sleep, los cuales permiten optimizar el consumo dependiendo del tipo de aplicación.

El chip integra un LP CPU (Low Power CPU) basado en arquitectura RISC-V, diseñado para ejecutar tareas simples mientras el resto del sistema se encuentra en reposo. Este núcleo puede permanecer activo incluso en estados de Deep Sleep, dependiendo de la configuración del sistema.

El sistema también incluye el RTC (Real Time Clock), el cual permite mantener el conteo de tiempo y gestionar eventos de despertar (wake-up) en modos de suspensión.

LIGHT SLEEP

El modo Light Sleep es un estado de bajo consumo en el cual el ESP32-C6 reduce significativamente su energía deteniendo la CPU principal, pero manteniendo activa la memoria RAM y parte del sistema interno.

En este modo:

- La CPU se detiene temporalmente.
- La memoria RAM permanece activa.
- El estado del programa se conserva.
- El sistema puede reanudar la ejecución rápidamente.
- El consumo energético se reduce de forma moderada.

El Light Sleep permite que el sistema “pause” su ejecución sin reiniciarse, lo que lo hace ideal para aplicaciones que requieren respuesta rápida después de un periodo de inactividad.

En esta práctica, el Light Sleep se implementa mediante MicroPython utilizando la función:

machine.lightsleep()

DEEP SLEEP

El modo Deep Sleep es el estado de menor consumo energético del ESP32-C6, en el cual la mayoría de los componentes internos se apagan completamente para minimizar el consumo de energía.

En este modo:

- La CPU principal se apaga completamente.
- La memoria RAM se desactiva.
- Solo el RTC y el LP Domain permanecen activos.
- El sistema pierde el estado de ejecución.
- El sistema reinicia al despertar.

Este modo es utilizado en sistemas embebidos donde es necesario reducir el consumo al máximo, especialmente en aplicaciones IoT o alimentadas por batería.

En esta práctica, el Light Sleep se implementa mediante MicroPython utilizando la función:

machine.deepsleep()

SISTEMA LP CPU Y RTC

El ESP32-C6 incluye un **LP CPU (Low Power CPU)** basado en arquitectura RISC-V, diseñado para operar con consumo mínimo de energía.

Este procesador puede:

- Ejecutar tareas simples en segundo plano.
- Mantenerse activo en modos de bajo consumo.
- Gestionar eventos de wake-up.

El sistema también utiliza el **RTC (Real Time Clock)**, que permite:

- Mantener el conteo de tiempo en Deep Sleep.
- Activar el despertar del sistema después de un tiempo definido.
- Gestionar eventos de temporización de bajo consumo.

DESPERTAR (WAKE-UP)

El ESP32-C6 puede despertar del modo de suspensión mediante diferentes mecanismos:

Timer (RTC Wake-up): despierta después del tiempo configurado. }

GPIO Wake-up: mediante cambios lógicos en pines (dependiendo del firmware).

LP CPU Wake-up: eventos generados por el procesador de bajo consumo.

Reset del sistema: en Deep Sleep, el sistema reinicia completamente.

ES IMPORTANTE RECALCAR QUE EL UNICO MODO EN MICROPYTHON ES PORMEDIO DE TIMER, EL RESTO DE LOS MODOS DE DESPERTAR AUN NO HAN SIDO IMPLEMENTADOS EN MICROPYTHON

7. Dependencias y Bibliotecas

Módulo/Biblioteca	Origen	Uso en el proyecto
machine	MicroPython stdlib	Permite el control del sistema de bajo consumo del ESP32-C6, incluyendo los modos Light Sleep y Deep Sleep
time	MicroPython stdlib	Control de retardos en la ejecución del programa y visualización de mensajes en consola

8. Consideraciones importantes.

Para el correcto funcionamiento de los modos de bajo consumo en el ESP32-C6 se deben tomar en cuenta las siguientes consideraciones:

- Utilizar micropython compatible con ESP32-C6 actualizado.
- Verificar que el firmware soporte lightsleep() y deepsleep().
- En Deep Sleep, el sistema se reinicia completamente al despertar.
- En Light Sleep, la memoria RAM se mantiene activa durante la suspensión.
- No todas las placas despiertan por GPIO en micropython.
- Utilizar tiempos adecuados en deepsleep(ms) para pruebas.
- El RTC es el único componente activo en Deep Sleep para control de temporización.
- El LP CPU puede permanecer activo en escenarios avanzados de bajo consumo.

9. Conclusiones

Esta práctica demuestra la implementación de los modos de bajo consumo en el ESP32-C6 utilizando micrpython, permitiendo comprender el comportamiento energético del microcontrolador en escenarios reales.

Principales resultados:

- El modo Light Sleep permite reducir el consumo energético sin perder el estado del sistema.

- El modo Deep Sleep apaga completamente el microcontrolador para minimizar el consumo.
- El sistema se reinicia automáticamente al salir de Deep Sleep.
- El uso del RTC permite controlar los tiempos de suspensión del sistema.
- El LP CPU forma parte de la arquitectura de bajo consumo del ESP32-C6.
- Estos modos son fundamentales para aplicaciones IoT y sistemas embebidos alimentados por batería.
- Micropython permite implementar estos modos de forma sencilla mediante la librería machine.

10. Bibliografía

1. Espressif Systems. (s. f.). *Low power mode on ESP32-C6 (BLE power management)*. ESP-IDF Programming Guide.
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-guides/low-power-mode/low-power-mode-ble.html>
2. Espressif Systems. (s. f.). *ESP32-C6 technical reference manual (ULP coprocessor section)*.
https://documentation.espressif.com/esp32-c6_technical_reference_manual_en.pdf#ulp
3. Espressif Systems. (s. f.). *esp_sleep_enable_timer_wakeup function documentation*. ESP-IDF API Reference.
https://docs.espressif.com/projects/esp-idf/en/stable/esp32c6/api-reference/system/sleep_modes.html#_CPPv429esp_sleep_enable_timer_wakeup8uint64_t
4. MicroPython. (s. f.). *machine module — RTC wake / low power features*.
<https://micropython-tve.readthedocs.io/en/esp32-low-power/library/machine.html>

11. Anexo

Código utilizado para entrar en modo Deep sleep con su modalidad timer, recordar que es el único modo que se ha implementado de momento en micropython para el ESP32 – C6.

```

import machine
import time
print("Sistema activo")

while True:

    print("Ejecutando tareas")

    time.sleep(2) #DEJAMOS UN DELAY PARA MOSTRAR MENSAJE

    print("Entra modo Deep sleep")

    machine.deepsleep(5000)

    #EL ESP32 SE REINICIA AL SALIR DE CUALQUIER MODO SLEEP.
    #DEBIDO A ESO ESTE MENSAJE NUNCA DEBE DE APARECER
    print("Mensaje que no debe de aparecer en consola")

```

Código utilizado para entrar en modo Light sleep con su modalidad timer, recordar que es el único modo que se ha implementado de momento en micropython para el ESP32 – C6.

```
Import machine
Import time
print("Sistema activo")

while True:

    print("Ejecutando tareas")

    time.sleep(2) #DEJAMOS UN DELAY PARA MOSTRAR MENSAJE

    print("Entra modo Light sleep")

    machine.lightsleep(5000)

    #EL ESP32 SE REINICIA AL SALIR DE CUALQUIER MODO SLEEP.
    #DEBIDO A ESO ESTE MENSAJE NUNCA DEBE DE APARECER
    print("Mensaje que no debe de aparecer en consola")
```